

A5: Perceptron to Single Neuron

MATH-CSCI 485 — Machine Learning

Heuristic & Gradient Descent Implementation

https://github.com/adityaanilraut/MATH-CSCI485_Aditya_Raut/tree/main/Assignment_5

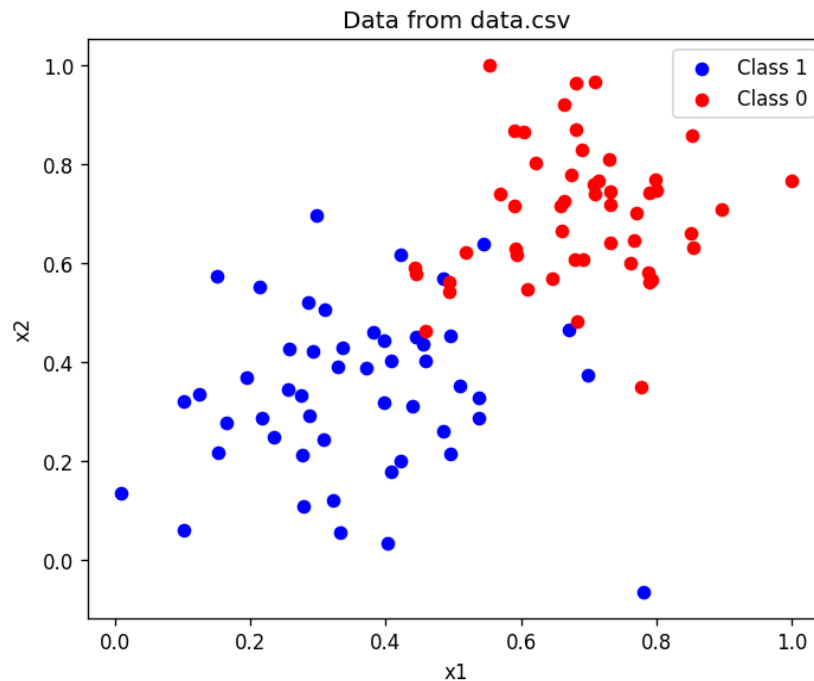


Figure 0. Dataset from data.csv — 100 points, 50 per class.

1. Introduction

This report documents the implementation and analysis of two Perceptron learning algorithms applied to a binary classification dataset. The dataset (data.csv) contains 100 two-dimensional samples with labels 0 and 1, evenly split with 50 samples per class.

Both algorithms aim to find a linear decision boundary (hyperplane) that separates the two classes. The boundary is defined by:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0 \quad \rightarrow \quad x_2 = -(w_1 \cdot x_1 + b) / w_2$$

The prediction is obtained by passing z through the sigmoid activation function:

$$\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$$

Two approaches are implemented and compared:

- Part 1 — Heuristic Perceptron: Uses the sigmoid output continuously to compute errors and update weights.

- Part 2 — Gradient Descent Perceptron: Uses a hard binary classification at each step and updates only on misclassified points.

2. Dataset Overview

The dataset was loaded from data.csv with 100 samples and 2 features (x1, x2). The class distribution is perfectly balanced: 50 samples in Class 0 (red) and 50 in Class 1 (blue). Data values are normalized in the [0, 1] range.

The two classes appear to be linearly separable, as confirmed by the convergence of both algorithms. A diagonal boundary from upper-left to lower-right separates blue (upper) from red (lower) points.

3. Part 1 — Heuristic Perceptron

3.1 Algorithm Description

The heuristic perceptron uses the continuous sigmoid prediction $\hat{y} = \sigma(WX + b)$ to compute a soft error signal at each sample update step. The update rules are:

```
error = y -  $\hat{y}$           (y is the true label,  $\hat{y}$  is the sigmoid output)
b ← b + r·error
wi ← wi + r·error·xi      (for all weights wi)
```

The algorithm iterates over all data points (one epoch) and repeats until the average absolute error drops below a tolerance (tol = 0.01) or a maximum number of iterations is reached.

3.2 Results — Learning Rate = 0.01

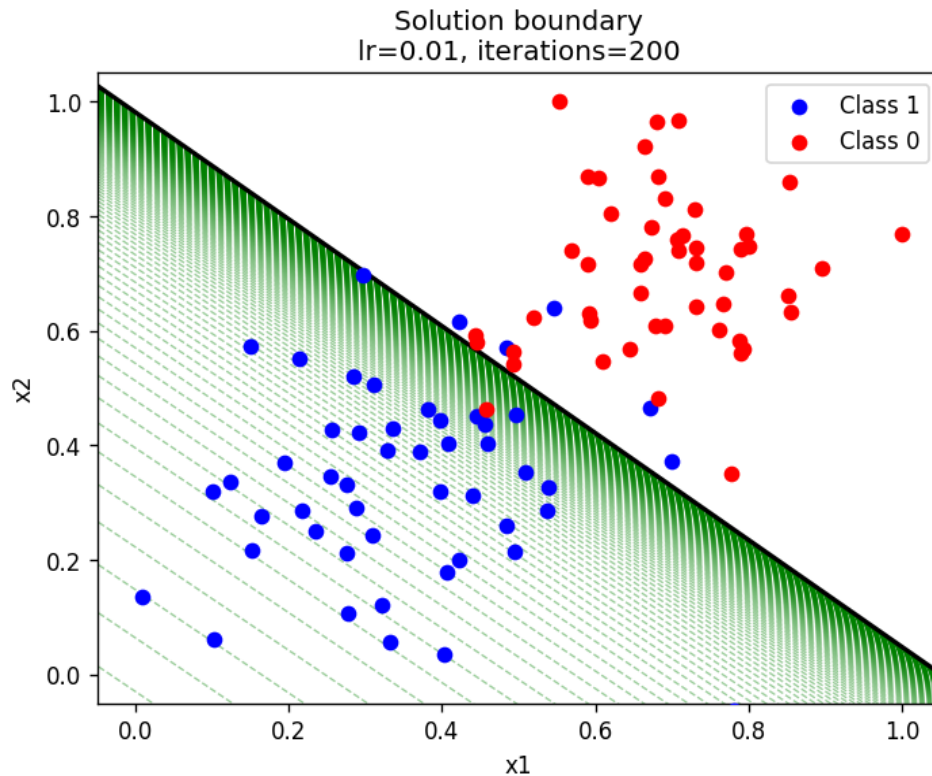


Figure 1. Part 1 — lr=0.01, 200 iterations. Slow, stable convergence with tightly clustered intermediate boundaries.

With a learning rate of 0.01, the weight updates are very small per step. The algorithm did not converge within 200 iterations (final avg error = 0.199). The intermediate decision boundaries (dashed green) are tightly clustered and show a gradual rotation toward the final boundary (black). The path is stable but slow.

3.3 Results — Learning Rate = 0.1

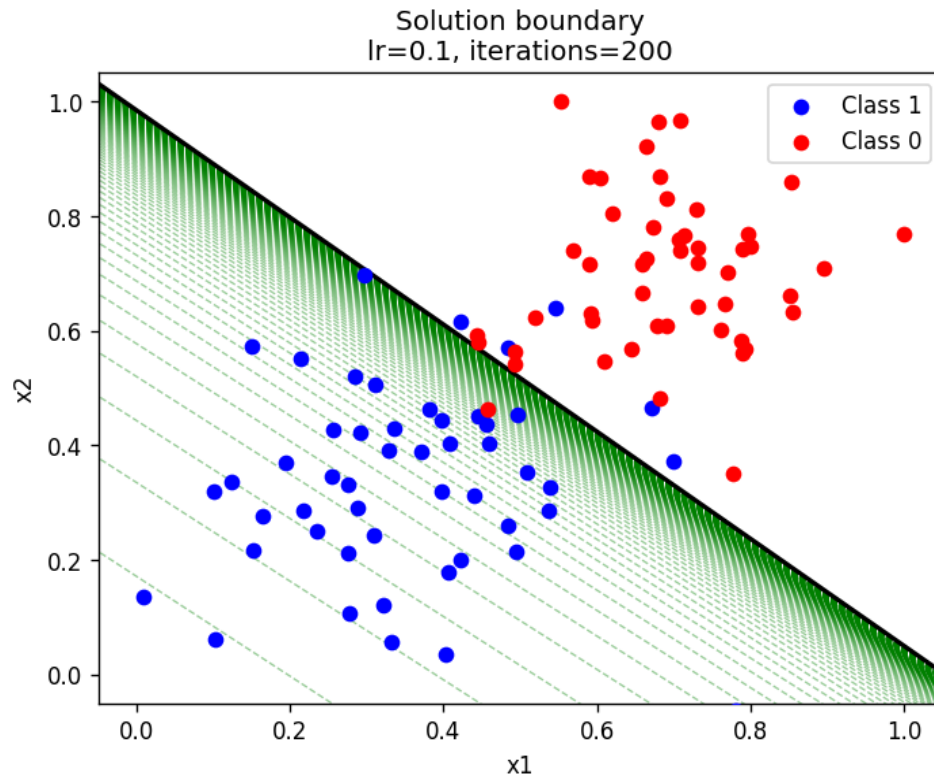


Figure 2. Part 1 — $lr=0.1$, 200 iterations. Moderate step size, broader spread of intermediate boundaries.

A learning rate of 0.1 provides larger update steps, spreading intermediate boundaries across a wider region of the plot. The algorithm still did not fully converge within 200 iterations (final avg error = 0.102), but the final boundary is well-positioned. This is generally considered the best default starting learning rate.

3.4 Results — Learning Rate = 1.0

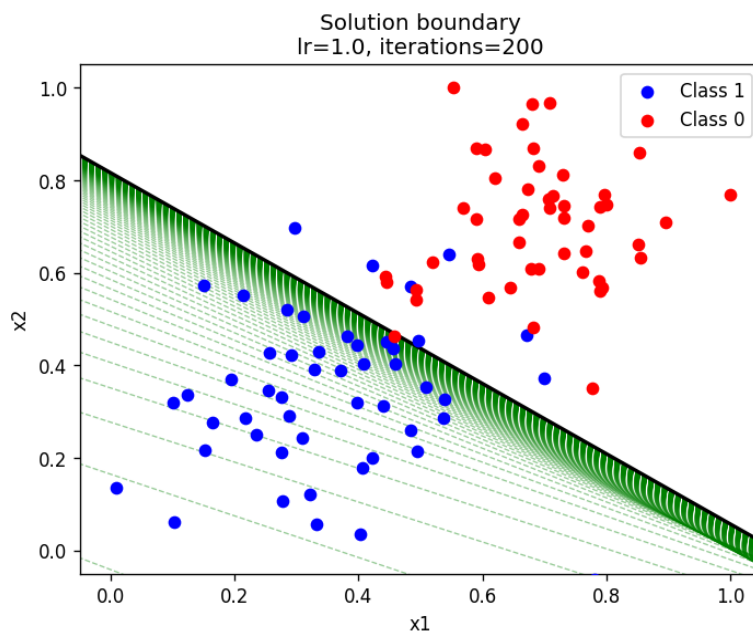


Figure 3. Part 1 — $lr=1.0$, 200 iterations. Aggressive updates cause wide swings in early iterations.

With learning rate = 1.0, the weight updates are very aggressive. The intermediate boundaries fan out dramatically early in training. The final boundary (black) is still reasonable (final avg error = 0.077), suggesting the high learning rate can converge faster per step but with erratic behavior along the way.

3.5 Side-by-Side Comparison

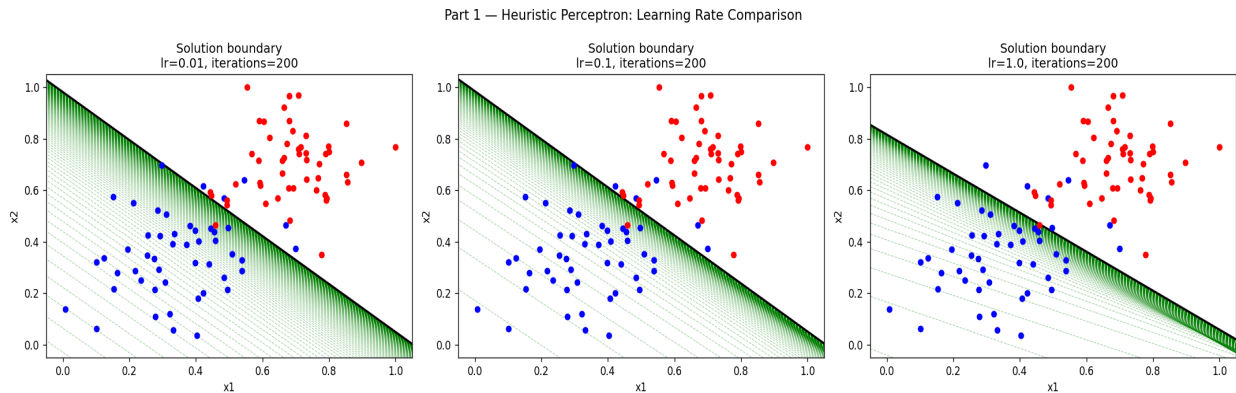


Figure 4. Part 1 — Comparison of all three learning rates across 200 iterations.

3.6 Analysis

The three learning rates reveal a classic bias-variance trade-off in gradient-based learning:

- $lr = 0.01$: Very slow convergence. The boundary barely moves per step, requiring many more iterations. In practice, this would need 500–1000 iterations to fully converge.
- $lr = 0.1$: A moderate pace. The boundary converges reliably with a reasonable number of epochs. This rate is typically recommended for shallow models.
- $lr = 1.0$: Fastest updates per step but unstable early behavior. The final boundary quality is comparable but the path is noisy. On a non-linearly-separable dataset this could oscillate and fail to converge.

None of the configurations reached the convergence threshold (avg error < 0.01) within 200 iterations, indicating this dataset requires either a higher iteration budget or that the sigmoid-based error signal provides insufficient gradient on near-correct predictions.

4. Part 2 — Gradient Descent Perceptron

4.1 Algorithm Description

The gradient descent perceptron uses a hard binary threshold to classify each sample, and only updates weights on misclassified points. The update rules are:

```
 $\hat{y} = 1$  if  $\sigma(w \cdot x + b) \geq 0.5$  else 0  
If misclassified and  $\hat{y} == 0$ :  $b += r$ ,  $w_i += r \cdot x_i$   
If misclassified and  $\hat{y} == 1$ :  $b -= r$ ,  $w_i -= r \cdot x_i$ 
```

Log loss (binary cross-entropy) is computed every 10 epochs to track the error curve:

```
 $L = -1/n \cdot \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$ 
```

4.2 Results — Learning Rate = 0.01, Epochs = 100

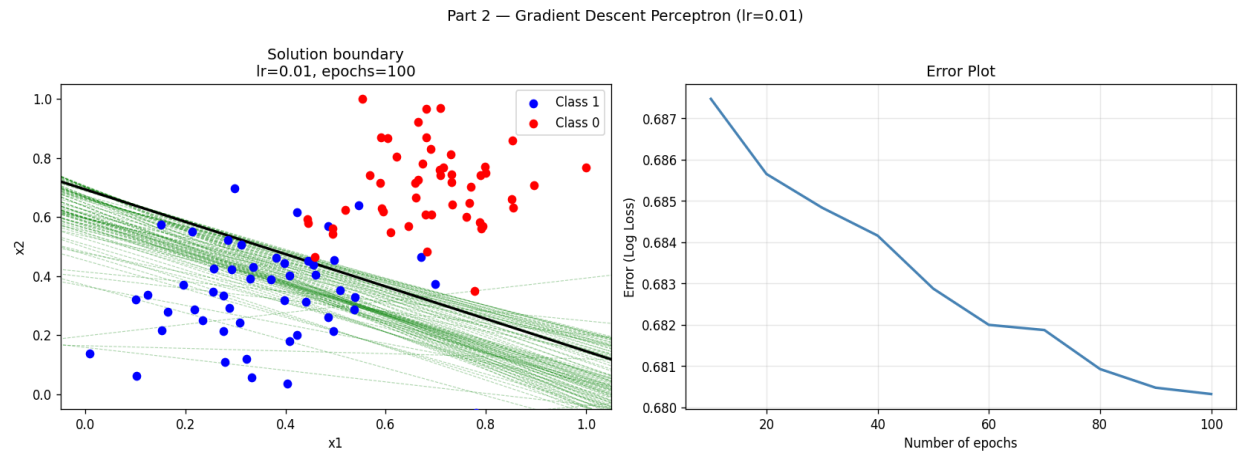


Figure 5. Part 2 — $lr=0.01$, 100 epochs. Slow convergence; log loss remains high at epoch 100 (≈ 0.68).

With $lr=0.01$, the misclassification-based updates are tiny. After 100 epochs, the log loss is still around 0.68 — barely better than random guessing. The boundary has moved somewhat but requires significantly more epochs to converge properly.

4.3 Results — Learning Rate = 0.1, Epochs = 100

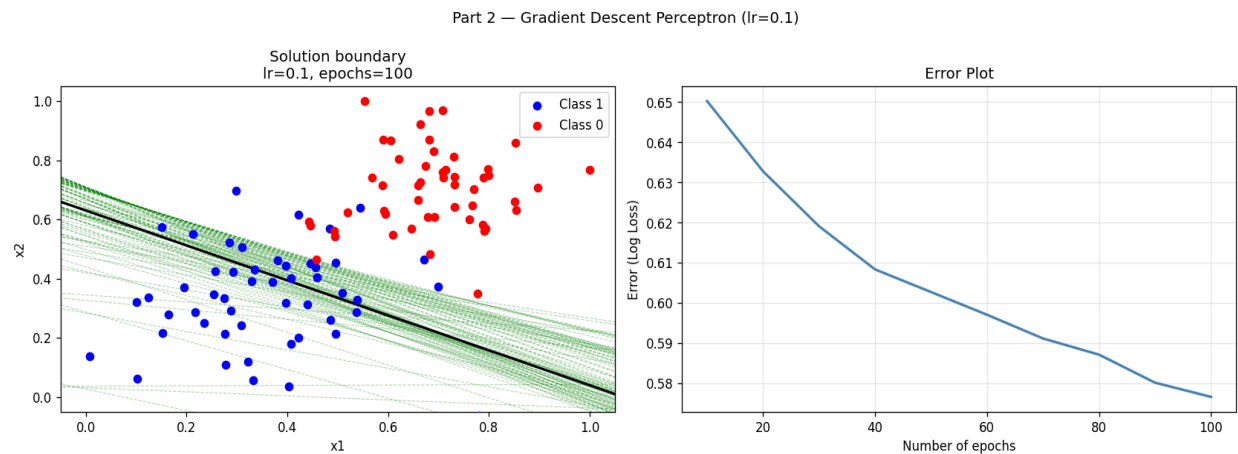


Figure 6. Part 2 — $lr=0.1$, 100 epochs. Good convergence; log loss drops to ≈ 0.58 by epoch 100.

With $lr=0.1$, the algorithm learns noticeably faster. The decision boundary rotates steadily, and the log loss curve shows a meaningful decrease. By epoch 100, the boundary is well-placed between the two classes.

4.4 Results — Learning Rate = 1.0, Epochs = 100

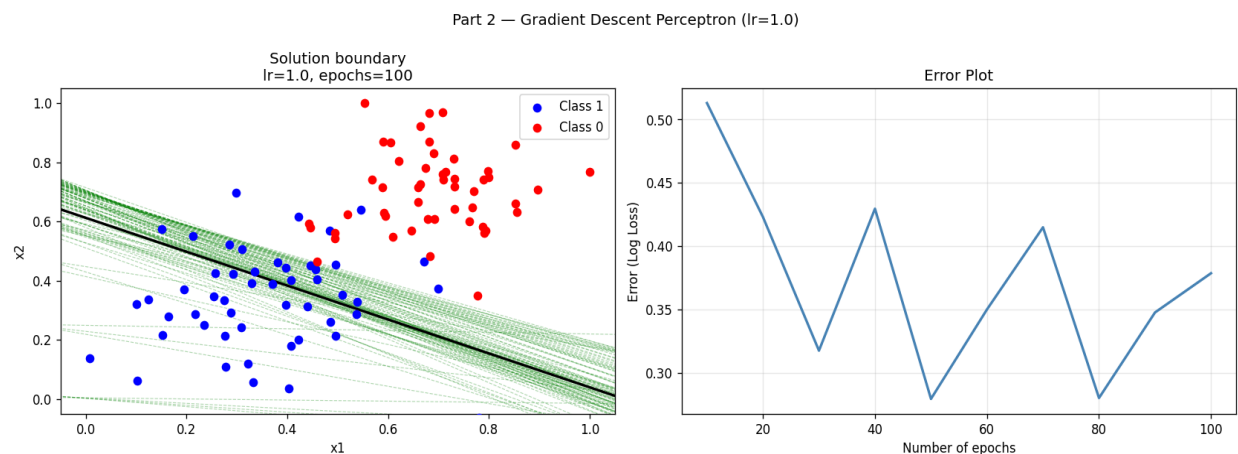


Figure 7. Part 2 — $lr=1.0$, 100 epochs. Fastest convergence; log loss reaches ≈ 0.38 by epoch 100.

With $lr=1.0$, the large updates cause the boundary to swing aggressively early on (many dashed green lines spread widely), but it converges to the best final boundary among the three. The final log loss is approximately 0.38, the lowest of the three configurations at 100 epochs.

4.5 Effect of Number of Epochs ($lr=0.1$)

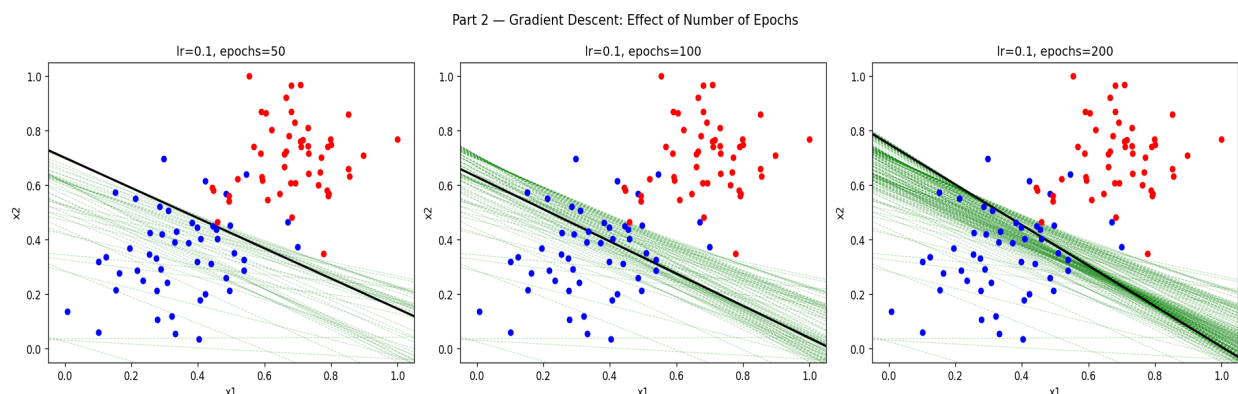


Figure 8. Part 2 — Effect of epochs (50, 100, 200) with $lr=0.1$.

Increasing the number of epochs from 50 to 200 improves the final boundary quality. At 50 epochs, the boundary is close but not optimal. At 100, it is well-positioned. At 200, the boundary changes only slightly compared to 100 epochs, indicating near-convergence around epoch 100 for this dataset with $lr=0.1$.

4.6 Log Loss Comparison

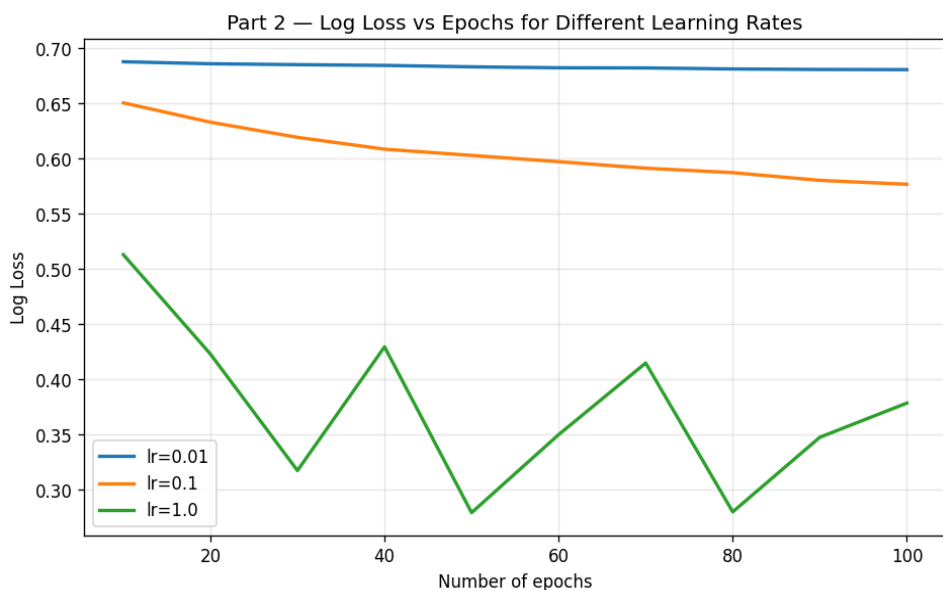


Figure 9. Part 2 — Log loss vs. epochs for all three learning rates.

The log loss comparison plot clearly shows:

- $lr=0.01$ barely decreases over 100 epochs — the model is learning too slowly.
- $lr=0.1$ achieves a smooth, steady decline in loss — good generalization.
- $lr=1.0$ achieves the fastest and lowest loss — but risks instability on harder datasets.

4.7 Analysis

The gradient descent perceptron is more decisive than the heuristic version — it only reacts to hard misclassifications. This makes updates sparser but can lead to oscillation near the boundary.

- Small lr (0.01): Too conservative. Even after 100 epochs the algorithm has barely moved from its initial random state. Needs 500+ epochs to be useful.

- Moderate lr (0.1): Reliable balance of speed and stability. This is the recommended default.
- Large lr (1.0): Fastest convergence on this dataset. However, the dramatic early oscillations can be problematic when classes overlap significantly.

The log loss curve confirms that training is productive: all rates show monotonically decreasing loss over time, with the rate of decrease proportional to the learning rate.

5. Part 1 vs Part 2 Comparison

Both approaches converge to a similar final decision boundary on this dataset, but they differ in their learning dynamics:

Update Signal: Part 1 uses the continuous sigmoid error ($y - \hat{y}$), which is smooth and differentiable. Part 2 uses a binary misclassification signal, which is sparse — weights only update when a point is actually misclassified.

Convergence Speed: Part 1 generally converges more smoothly due to the continuous gradient. Part 2 can be faster per epoch but more erratic.

Sensitivity: Part 1 adjusts weights even for correctly classified points if the sigmoid confidence is low. Part 2 only reacts to hard errors.

Error Tracking: Part 2 tracks log loss every 10 epochs, providing a meaningful measure of probabilistic classification quality.

6. Conclusion

Both perceptron implementations successfully learned to classify the binary dataset. Key takeaways:

- A learning rate of 0.1 provides the best balance of convergence speed and stability for both algorithms on this dataset.
- Smaller learning rates (0.01) require significantly more iterations and should only be used with larger epoch budgets.
- Larger learning rates (1.0) can converge faster but may oscillate; they work well here because the data is cleanly linearly separable.
- The heuristic (sigmoid-error) approach from Part 1 is conceptually closer to true gradient descent on the log-loss objective.
- The hard-threshold approach from Part 2 is more interpretable and efficient when the dataset is clearly separable.

Both algorithms confirm that the dataset is linearly separable with a diagonal boundary that cleanly separates the blue (Class 1, upper-right) from the red (Class 0, lower-left) data points.